You are working in the Hydrion Flutter repository.

Your task is to perform a repository-wide cross-platform audit and implementation pass so Hydrion remains a genuine Android and iOS Flutter product rather than becoming Android-only through incremental development.

This is not a documentation-only task and it is not permission to regenerate every platform folder blindly.

Do not stage, commit, push, create branches, open pull requests, publish builds, or modify remote resources.

Do not begin by editing files.

First inspect the complete repository and establish the real current state. Every conclusion must be grounded in source code, configuration, tests, generated platform projects, CI workflows, dependency metadata, and existing documentation.

Core product requirement

Hydrion V1 is a shared Flutter mobile product intended to support:

- Android
- iOS

Android APK availability is only the current distribution path. It must not be interpreted in code, documentation, architecture, or release planning as evidence that Hydrion is an Android-only product.

Existing web or desktop targets may remain supported where they already work, but they must not weaken Android or iOS quality or cause mobile-specific functionality to be falsely represented as universally available.

The application’s primary product logic must remain shared Flutter and Dart code.

Known environment limitation

The current development environment may not provide:

- macOS
- Xcode
- CocoaPods
- an Apple Developer account
- Apple signing certificates

- provisioning profiles
- an iPhone
- an iOS simulator
- TestFlight access

Therefore:

- perform every repository-level iOS compatibility check possible in the current environment
- prepare all source-controlled iOS configuration that can be prepared safely
- isolate or correct Android-only assumptions
- create credible future macOS validation and CI paths
- document all unverified iOS behavior explicitly
- never claim that iOS compilation, signing, device installation, TestFlight distribution, or App Store readiness was completed without evidence

A missing Mac is a validation limitation, not permission to ignore iOS architecture.

Mandatory execution gates

Follow these gates in order.

Gate 1: Repository audit

Inspect the complete repository before making changes.

Produce an internal audit covering:

- Flutter project structure
- `pubspec.yaml`
- dependency lockfile
- application entry points
- routing and navigation
- services
- repositories
- platform abstractions
- permissions
- local persistence
- notifications and reminders
- location and weather
- profile-photo handling
- lifecycle behavior
- Android project configuration
- iOS project configuration
- web and desktop targets

- tests
- CI workflows
- release scripts
- signing configuration
- documentation
- product backlog and user stories

Do not infer implementation from documentation alone. Verify runtime source.

Gate 2: Current support classification

Before editing, classify every relevant capability as one of:

- verified shared implementation
- verified Android implementation
- verified iOS implementation
- configured but unvalidated
- partially implemented
- platform-dependent
- missing
- blocked by current environment
- documented only
- obsolete or misleading

At minimum classify:

- hydration logging
- history
- analytics
- reminders
- notifications
- location permissions
- weather retrieval
- weather-adjusted goals
- profile images
- local storage
- challenges
- startup behavior
- external links
- accessibility
- localization
- networking
- background behavior
- app metadata

- release signing

Gate 3: Proposed change plan

After the audit, formulate a repository-grounded implementation plan.

The plan must identify:

- exact problems
- affected platforms
- affected files
- proposed abstractions
- configuration changes
- tests required
- CI changes
- documentation changes
- risks
- work that cannot be validated without macOS

Only then begin implementation.

Do not pause for approval unless a change would be destructive, would require replacing a major dependency, would alter user data formats incompatibly, or would require credentials not present in the repository.

Audit and implementation requirements

1. Flutter platform structure

Inspect these targets where present:

- `android/`
- `ios/`
- `web/`
- `windows/`
- `macos/`
- `linux/`

Confirm:

- whether each folder exists
- whether it appears valid for the repository's current Flutter version
- whether it contains stale package identifiers or default placeholders
- whether generated files are consistent with current dependencies

- whether Android-focused work has broken or neglected iOS
- whether any platform target is claimed in documentation but absent or unusable

Do not run `flutter create .` or regenerate platform folders unless there is clear evidence that regeneration is necessary and safe.

Do not modify unrelated desktop targets merely for cosmetic consistency.

Hydrion’s Android application ID is intended to be:

```text
com.the1807.hydrion
```

Determine the correct future iOS bundle identifier without inventing Apple account values. Use a repository-safe documented placeholder only where external Apple configuration is still required.

2. Dependency compatibility

Audit every dependency and dev dependency in pubspec.yaml.

For every production dependency, record:

purpose in Hydrion

Android support

iOS support

web or desktop implications where relevant

required Android configuration

required iOS configuration

minimum platform requirements

native permissions or capabilities

maintenance status

known platform limitations

whether Hydrion currently initializes it safely

whether it is used behind a testable abstraction

whether fallback behavior exists

Pay particular attention to dependencies associated with:

flutter_local_notifications

location and geolocation

weather networking

permissions

image_picker

shared preferences

file-system storage

secure storage, if present

timezone handling

background scheduling

connectivity

URL launching

package metadata

device information

analytics

crash reporting

authentication

accessibility

localization

HTTP networking

Do not replace a dependency merely because another package exists.

Replace or isolate a dependency only when justified by:

missing iOS support

security risk

abandonment

incompatible licensing

unresolvable platform behavior

inability to test or maintain it safely

If online package metadata cannot be verified in the current environment, state that limitation rather than guessing.

3. Android-specific assumptions

Search the entire repository for:

Kotlin

Java

Gradle-specific assumptions

MethodChannel

EventChannel

Platform.isAndroid

Android-only imports

Android SDK APIs

Android-only lifecycle handling

Android-only paths

external-storage assumptions

Android-specific notification code

Android-specific background services

URI and intent handling

Android-specific permission logic

Android-specific build flags

Android-only plugin initialization

platform checks without an iOS branch

hard-coded package names

hard-coded file-system separators

assumptions about APK distribution

Identify every essential feature that is currently Android-dependent.

For each one:

implement the iOS equivalent where repository-level work is possible

otherwise isolate it behind a clear platform interface

provide an explicit safe fallback

surface platform limitations honestly

prevent silent feature disappearance on iOS

Do not scatter new Platform.isAndroid and Platform.isIOS checks throughout widgets and business logic.

4. Platform-service architecture

Ensure platform-specific behavior is isolated behind stable Dart interfaces.

The architecture should keep:

hydration calculations in shared Dart

weather-goal calculations in shared Dart

persistence contracts in shared Dart

notification scheduling behind a notification service

location access behind a location service

profile-photo access behind a media service

permissions behind explicit service boundaries where appropriate

platform differences centralized

native-plugin failures recoverable

unit tests independent of native plugin initialization

Do not over-engineer abstractions that have only one trivial use.

Prefer small, explicit interfaces with production adapters and test fakes.

Unsupported or unavailable operations must fail gracefully with actionable user-facing states rather than uncaught plugin exceptions.

5. iOS project readiness

Audit and update the iOS project where repository-level changes are justified.

Inspect:

ios/Runner/Info.plist

ios/Runner/AppDelegate.*

ios/Runner.xcodeproj

ios/Runner.xcworkspace assumptions

ios/Podfile

deployment target

bundle identifier configuration

display name

supported orientations

notification capability requirements

background-mode requirements

location permission descriptions

photo-library permission descriptions

camera permission descriptions, only if camera access is actually supported

URL schemes

deep links

App Transport Security

localization declarations

privacy usage strings

generated plugin registrant assumptions

launch-screen configuration

app-icon requirements

privacy manifest requirements

required entitlements

Do not add:

fake Apple Team IDs

fake certificates

fake provisioning profiles

fake App Store identifiers

fake Apple credentials

secrets

developer-specific absolute paths

Where Apple-side configuration cannot be committed safely, document the exact required external step.

Permission descriptions must accurately describe Hydrion’s real behavior and must not request access the app does not use.

6. Notifications and reminders

Hydrion reminders are a core feature. Audit them as a cross-platform product capability, not merely as an Android plugin call.

Verify or prepare:

permission request timing

permission rationale

denied permission behavior

permanently denied behavior

iOS provisional or standard authorization behavior where applicable

Android notification-channel creation

Android runtime notification permission behavior

foreground notification behavior

background behavior

terminated-app behavior

schedule persistence

schedule restoration after restart

cancellation

rescheduling

duplicate prevention

timezone initialization

daylight-saving transitions

invalid or past schedules

notification identifiers

reminder edits

reminder deletion

app upgrades

platform capability limitations

Do not assume Android background scheduling semantics apply to iOS.

Do not promise exact delivery where the operating system may defer notifications.

Clearly document platform limitations in user-facing behavior, technical documentation, and testing guidance.

7. Location and weather

Audit the complete weather-personalization path.

Confirm:

location is requested only after a user chooses weather personalization

refusal does not break hydration logging

users can continue with standard hydration goals

disabled location services are handled

unavailable coordinates are handled

approximate location is acceptable where supported

stale weather data is identified

weather-network failures have a safe fallback

location data is not persisted or transmitted unnecessarily

weather requests are not coupled to Android APIs

Android permissions are correct

iOS usage descriptions are present

privacy and legal copy accurately reflect actual data flow

the UI distinguishes base goals from weather adjustments

no weather-derived goal is presented without explaining its source

Add tests for permission denial, unavailable services, stale data, network failure, and fallback hydration goals.

8. Profile photos and local files

Audit the local-photo feature introduced through image_picker.

Confirm:

Android selection works through supported system APIs

iOS selection is configured correctly

permission strings match actual behavior

camera permission is not requested unless camera capture exists

selected images are persisted in an app-controlled location

temporary picker paths are not treated as permanent storage

missing or deleted image files are handled

replacing a photo cleans up obsolete app-owned files where safe

removing a photo restores the shark-avatar fallback

no Android absolute path is stored as a portable identity

file operations use cross-platform APIs

tests do not require a live native picker

Do not access unrestricted external storage.

Do not introduce broad photo-library permissions where a modern system picker can avoid them.

9. Local persistence and migration

Audit persisted data for:

hydration logs

settings

reminders

challenges

profile information

profile-photo references

weather preferences

achievements

onboarding completion

application version migrations

Confirm that storage does not depend on:

Android external-storage paths

removable storage

Android directory names

unsupported iOS paths

Windows-style path separators

temporary directories for durable data

Preserve existing user data.

If a data-format migration is needed:

make it backward-compatible where practical

test it

document it

never silently erase data

10. iPhone interface compatibility

Audit the existing Hydrion UI without replacing its visual identity.

Check all major screens, including:

startup

onboarding

Home

Challenges

Progress

Coach

Profile

profile editing

Settings

reminders

legal pages

dialogs

bottom navigation

Bottle Bingo

image-picker states

weather states

Validate:

SafeArea behavior

status-bar overlap

Dynamic Island and notch regions

home-indicator overlap

keyboard insets

small iPhone widths

large iPhone screens

portrait assumptions

landscape behavior where supported

text scaling

accessibility text sizes

scrollability

overflow

bottom-sheet sizing

dialog sizing

touch targets

image scaling

bottom-navigation labels

platform-consistent back behavior

focus and semantics

reduced motion

Do not perform another broad redesign.

Preserve the current Hydrion character-led visual system while correcting layouts that are unsafe or unusable on iOS.

Use adaptive or Cupertino behavior only where it materially improves platform correctness. Do not make Android and iOS look like unrelated products.

11. Tests

Add or update tests for cross-platform behavior.

The standard test suite must not require a real device or native plugin host.

Cover where applicable:

service abstractions

Android and iOS platform branches

unsupported-platform fallback

plugin initialization failure

notification permission granted

notification permission denied

permanently denied states

reminder creation

reminder editing

reminder cancellation

duplicate prevention

timezone handling

daylight-saving-sensitive calculations

location denial

location services disabled

weather failure

standard-goal fallback

profile-photo selection abstraction

missing persisted profile image

profile-image removal

local-data persistence

small-screen rendering

large text scaling

safe-area-sensitive layouts

bottom-navigation behavior

platform back navigation

iOS configuration expectations that can be statically validated

Android configuration expectations

existing hydration behavior remaining intact

Separate unit/widget tests from device or simulator integration tests.

Document native integration cases that require:

Android device or emulator

iOS simulator

physical iPhone

notification delivery over time

permission dialogs

app termination and relaunch

12. Validation

Run every validation genuinely supported by the current environment.

At minimum attempt, where available:

flutter pub get
dart format --set-exit-if-changed .
flutter analyze
flutter test
flutter build web --release
flutter build apk --release
flutter build appbundle --release
flutter doctor -v

Also validate:

YAML syntax

CI workflow syntax

repository formatting

dependency configuration

Android manifest

Gradle configuration

iOS plist syntax

iOS Podfile consistency

platform-specific configuration files

absence of committed signing secrets

absence of developer-specific absolute paths

absence of keystores and Apple credentials

Do not claim an Android build passed if the Android SDK is unavailable.

Do not run or claim:

flutter build ios

as successful outside a valid macOS and Xcode environment.

Record every command, exit result, and environment limitation.

13. CI strategy

Audit all current GitHub Actions workflows.

The repository must not claim cross-platform support while validating only Android.

Design clear CI responsibilities:

Shared validation

Run on an appropriate runner:

dependency resolution

formatting

static analysis

unit tests

widget tests

repository checks

web build where still supported

Android validation

Run separately:

debug or unsigned validation where appropriate

release APK validation

app bundle validation

signed release artifact only when protected signing secrets exist

no signing secrets exposed in logs or artifacts

iOS compatibility validation

Add or design a macOS-runner job where justified:

Flutter setup

CocoaPods setup

dependency resolution

static analysis and tests if not already shared

flutter build ios --simulator or another credible unsigned compatibility build

flutter build ios --no-codesign where appropriate for device-target compilation

iOS configuration validation

Use the exact build mode that is valid for the selected runner and explain why.

Routine iOS compatibility CI must not depend on distribution certificates where unsigned validation is sufficient.

Protected iOS release

Keep separate from routine CI:

Apple signing

archive

App Store Connect authentication

TestFlight upload

release approval

Do not add fake secrets.

Document required future GitHub secrets and Apple-side configuration without inserting values.

Avoid wasting CI minutes through redundant builds when existing jobs can be reused safely.

14. Documentation truthfulness

Update repository documentation to distinguish clearly:

Product support

Hydrion is designed as a Flutter mobile product for Android and iOS.

Current Android distribution

Signed Android beta artifacts may currently be available through the repository's release process.

Current iOS status

Repository-level iOS compatibility may be prepared, but final compilation, signing, physical-device validation, TestFlight distribution, App Store Connect setup, and Apple review require macOS, Xcode, Apple Developer configuration, and Apple-controlled services.

Do not describe Android APK availability as the complete Hydrion V1 release.

Do not use "iOS supported" without qualifying whether that means:

architecturally supported

repository configured

CI compiled

simulator tested

physical-device tested

TestFlight tested

App Store released

15. iOS readiness document

Create or update a dedicated repository document for the future Mac-based validation pass.

It must cover:

supported macOS requirements

supported Xcode requirements

Flutter version alignment

flutter doctor -v

Xcode command-line tools

CocoaPods installation

pod install

bundle identifier

Apple Developer Team

signing certificates

provisioning profiles

entitlements

notification capability

background modes

location permissions

photo-picker behavior

app icons

launch screen

privacy manifest

Info.plist usage descriptions

simulator validation

small and large iPhone layouts

physical-device validation

permission prompts

notification delivery

background and terminated-app behavior

release build

archive creation

App Store validation

TestFlight upload

internal testing

external testing

export-compliance questions

App Store privacy questionnaire

metadata

screenshots

review submission

rollback

build expiration

tester
Paste this into Codex:

You are working in the Hydrion Flutter repository.

Your task is to redesign Hydrion into a polished, modern, character-led mobile product with the visual confidence of high-quality health, wellness, social, and gamified mobile applications.

This is not a light restyling pass.

Do not merely change colors, increase border radius, add gradients, or rearrange existing settings cards.

The current app must be redesigned at the product-experience level so it feels intentional, premium, expressive, useful, and alive.

Do not stage, commit, push, create branches, publish builds, or release anything.

Do not begin changing files immediately.

First audit the current repository, navigation, screen structure, theme system, widgets, assets, shark characters, profile system, weather logic, reminders, hydration logging, challenges, analytics, legal pages, startup flow, localization, accessibility, tests, and platform support.

Then produce an implementation plan and execute it.

Product objective

Hydrion must stop feeling like:

- a settings application
- a collection of generic Flutter cards
- a prototype made from default Material widgets
- repeated onboarding forms
- a hydration calculator with a mascot pasted on top

Hydrion should feel like:

- a modern hydration companion
- a wellness dashboard
- a character-led daily ritual
- a lightly gamified progress experience
- a product with a recognizable visual identity
- an app users would proudly share in screenshots
- an app that feels equally intentional on Android and iOS

The target quality is comparable to polished health, wellness, social, fitness, and gamified profile applications.

Do not copy any external design directly.

Use the following design principles as inspiration only:

- Instagram-like navigation simplicity
- health-app dashboard clarity
- gamified profile identity
- large character-led hero sections
- strong visual hierarchy
- clean typography
- expressive but controlled color
- visible progress
- collectible achievements
- social-product confidence
- polished mobile spacing
- minimal but meaningful interaction

Non-negotiable design principles

1. Strong visual hierarchy

Every screen must have:

- one obvious primary purpose
- one dominant focal area
- clear primary and secondary actions
- consistent content grouping
- deliberate typography scale
- balanced whitespace
- meaningful use of color
- fewer competing surfaces

Do not show ten equally weighted cards on one screen.

Do not give every section the same visual prominence.

2. Hydrion-specific identity

The UI must feel unmistakably like Hydrion.

Build around:

- ocean depth
- water movement

- hydration progress
- underwater light
- droplets
- bubbles used sparingly
- shark personality
- aquatic gradients
- cool neutrals
- clean white space
- strong blue and cyan accents
- occasional contextual accent colors for weather, streaks, warnings, and achievements

Avoid making every screen fully blue.

Use color with hierarchy.

3. Character-led experience

The shark companion must be an active product element, not decorative clip art.

The shark should respond to:

- time of day
- hydration progress
- being behind pace
- being on track
- nearing goal
- completing goal
- weather conditions
- streaks
- challenge progress
- missed days
- reminder interactions
- milestone completion

The shark may change:

- pose
- supporting asset
- expression where assets allow
- speech
- background treatment
- companion card state
- animation intensity
- surrounding environmental details

Do not use constant motion.

Use subtle, intentional animation.

Respect reduced-motion preferences.

4. Fewer generic cards

Do not solve every screen with identical rounded rectangles.

Use a mix of:

- hero regions
- progress rings
- charts
- stat clusters
- horizontal collections
- segmented controls
- large media areas
- compact list rows
- challenge tiles
- badge collections
- modal sheets
- profile identity headers
- contextual panels
- empty states
- celebration states

Every surface must have a reason to exist.

5. Shared design system

Create or refine reusable Hydrion design tokens for:

- color
- typography
- spacing
- radii
- shadows
- elevation
- icon sizing
- motion duration
- easing

- surface types
- semantic states
- gradients
- chart styling
- button hierarchy
- navigation styling

Do not hard-code arbitrary values across screens.

Required screen architecture

1. App shell and navigation

Create a polished mobile shell with bottom navigation.

Primary destinations:

- Home
- Challenges
- Progress
- Coach
- Profile

Requirements:

- clear active state
- strong iconography
- compact labels
- correct SafeArea handling
- no overlap with Android navigation or iPhone home indicator
- preserved tab state where appropriate
- smooth but restrained transitions
- no generic default NavigationBar appearance
- consistent visual treatment across Android and iOS
- accessible labels and touch targets

The profile avatar should be available in a logical top-level location where appropriate.

Settings must be secondary, not a primary tab.

2. Home dashboard

Redesign Home around one strong daily hydration story.

The user should immediately understand:

- how much they have consumed
- today's target
- how much remains
- whether they are on pace
- what affected today's goal
- what to do next
- how the shark companion is responding

Use a visually dominant hero area containing:

- shark companion
- daily progress
- current status
- contextual message
- primary quick-log action

Additional content may include:

- weather adjustment
- streak
- next reminder
- active challenge
- recent hydration
- today's timeline
- achievement progress

Do not create a long vertical settings page.

Use horizontal collections where useful.

Make the logging action extremely clear.

Quick-log amounts should be easy to reach without opening several screens.

3. Hydration progress visualization

Create a visually strong, branded hydration progress system.

Possible forms:

- liquid ring
- vertical fill

- circular progress
- bottle fill
- wave-based progress
- timeline
- segmented daily progress

Requirements:

- current intake
- goal
- percentage
- remaining amount
- over-goal behavior
- accessible text equivalent
- reduced-motion behavior
- readable on small screens
- safe at large text sizes

Do not rely on visual progress alone.

4. Weather personalization

Weather-enabled users must visibly experience a personalized dashboard.

Show:

- current weather
- location label at an appropriate privacy level
- base hydration goal
- weather adjustment
- final daily goal
- plain-language reason
- updated time
- fallback state

Example:

- Base goal: 2.4 L
- Weather adjustment: +350 mL
- Today’s goal: 2.75 L
- Reason: Warm conditions in Windsor

Weather-disabled users should see:

- standard goal
- clear explanation
- optional invitation to enable weather personalization
- no fake weather data
- no silent hidden adjustment

Handle:

- denied permission
- disabled location service
- stale weather
- unavailable network
- incomplete weather response
- estimated fallback
- location privacy

5. Profile dashboard

Create a real profile destination.

Do not reuse onboarding.

Do not make users feel they are creating an account again.

Profile should include:

- profile photo or shark avatar
- display name
- hydration identity
- streak
- achievements
- challenges completed
- consistency score
- preferred units
- weather personalization status
- reminders summary
- progress summary
- badges
- edit profile
- settings
- support
- legal access

The profile header should feel expressive and personal.

Use profile identity as part of the product experience.

Support:

- selecting a local photo
- replacing the photo
- removing the photo
- retaining the photo locally
- fallback to shark avatar
- handling missing image files
- handling permission denial
- handling picker cancellation

The shark companion must remain part of the product even if the user selects a personal image.

6. Profile menu

Where the profile image appears in the top interface, allow a compact menu or dropdown with:

- View Profile
- Settings
- Help and Support
- About Hydrion

Do not include Sign Out unless real authentication exists.

Do not create fake account behavior.

7. Progress dashboard

Redesign Progress as a real analytics destination.

Include meaningful visualizations such as:

- weekly hydration consistency
- daily intake chart
- current streak
- longest streak
- average completion
- best day
- missed-goal patterns
- weather impact summary
- challenge contribution

- achievements

Use charts and stat groupings carefully.

Do not overload the screen.

Provide clear time filters where justified:

- week
- month
- longer-term period if supported

Use accessible chart labels and summaries.

8. Challenges

Create an engaging challenge experience.

Challenge list should show:

- title
- visual identity
- status
- duration
- progress
- reward or badge
- difficulty
- next action

Challenge details should show:

- purpose
- rules
- progress
- daily steps
- current streak
- completion state
- shark reaction
- safety note where needed
- reward

Bottle Bingo must be interactive.

Requirements:

- real grid
- tappable cells
- persisted completion state
- clear completed state
- progress count
- completion celebration
- reset behavior only with confirmation
- accessible labels
- safe hydration prompts

Do not include unsafe hydration behavior.

9. Coach

The Coach destination must feel conversational and useful.

It should include:

- shark companion state
- today’s recommendation
- weather-aware guidance
- progress-aware advice
- reminder suggestions
- challenge recommendations
- recovery messaging after missed goals
- calm safety messaging

Do not present the coach as an unrestricted medical expert.

Do not use generic chatbot styling unless real conversational behavior exists.

10. Settings

Settings should become a clean secondary utility screen.

Group settings into clear categories:

- Profile
- Hydration preferences
- Units
- Weather
- Notifications
- Accessibility

- Language
- Data
- Legal
- Support

Avoid placing all settings directly on Home.

Use standard mobile settings patterns where appropriate, but keep Hydrion branding.

11. Legal and policy experience

The current legal pages must feel serious and professionally structured.

Create clearly separated sections for:

- Terms and Conditions
- Privacy Policy
- Health and Safety Disclaimer
- Weather and Location Data
- Notifications
- Local Storage
- Profile Photos
- User Responsibilities
- Data Deletion
- Third-Party Services
- Limitations
- Support Contact
- Effective Date
- Document Version

Requirements:

- readable typography
- table of contents or section navigation
- clear headings
- sufficient detail
- honest description of actual app behavior
- draft pending professional review disclaimer
- no fake legal claims
- no fake regulatory compliance
- no fake certifications
- no medical claims

State clearly:

- Hydrion is not medical advice
- hydration recommendations are estimates
- excessive water intake can be harmful
- users with health conditions should seek professional advice
- weather and location are optional
- profile photos remain local if that is the actual implementation
- data behavior must match reality

12. Startup and loading

Redesign the startup experience so it feels polished.

Requirements:

- quick first response
- no long empty screen
- no awkward oversized mascot
- no cheap looping animation
- no generic spinner
- no progress bar that feels disconnected from startup work
- smooth handoff into Home or onboarding
- real progress where available
- reduced-motion support
- low-end Android performance
- iPhone-safe layout

Use the shark and droplet as part of one coherent composition.

If the existing droplet loader logic is usable, preserve the logic while improving:

- scale
- composition
- timing
- easing
- glow
- wave movement
- shark movement
- completion transition

13. Onboarding

Keep onboarding distinct from Profile.

Onboarding should only collect what is needed for initial setup.

It should feel:

- welcoming
- short
- visual
- clear
- progressive
- optional where appropriate

Do not force users to repeatedly configure the app.

Do not reuse onboarding for later profile editing.

14. Responsive behavior

The redesign must work across:

- small Android phones
- large Android phones
- iPhone SE-sized screens
- modern iPhones
- notched devices
- Dynamic Island devices
- large text settings
- reduced motion
- portrait mode
- keyboard-open states

Check:

- overflow
- SafeArea
- scroll behavior
- bottom navigation
- image clipping
- modal sizing
- chart sizing
- text wrapping
- action reachability

Do not optimize only for one screenshot size.

Asset strategy

Audit all shark and brand assets.

Use existing assets deliberately.

Do not rename assets arbitrarily.

Do not invent false character identities.

Create a documented mapping between:

- asset filename
- shark identity
- intended emotional state
- intended use

If existing PNG assets limit animation, use:

- scale
- opacity
- tilt
- translation
- parallax
- glow
- background motion
- bubbles
- wave motion

Do not fake facial animation that the art does not support.

Do not add Rive, Lottie, or another animation dependency unless there is a strong repository-grounded reason.

Flutter engineering requirements

Use reusable widgets for repeated product patterns.

Possible reusable components include:

- HydrionShell
- HydrionBottomNavigation
- SharkCompanionHero
- HydrationProgressHero

- WeatherAdjustmentPanel
- ProgressMetricTile
- ChallengeCard
- BadgeCollection
- ProfileIdentityHeader
- HydrionSectionHeader
- HydrionPrimaryButton
- HydrionSurface
- HydrionEmptyState
- HydrionErrorState
- HydrionLegalSection
- HydrionAvatarMenu

Names may vary based on existing conventions.

Do not create a monolithic screen file.

Do not duplicate styling.

Preserve existing services, repositories, persistence, and domain logic unless change is necessary.

Keep business logic out of widgets.

Ensure native plugins remain testable through abstractions.

Cross-platform requirement

Hydrion must remain a shared Flutter application for Android and iOS.

Do not introduce:

- Android-only paths
- Android-only UI assumptions
- Android-only photo behavior
- Android-only notification behavior
- Android-only navigation behavior
- Android-specific code inside shared widgets

Where platform adaptation is needed, use safe Flutter abstractions.

Do not claim iOS validation without a Mac and Xcode.

Accessibility

Audit and implement:

- semantic labels
- touch targets
- contrast
- text scaling
- screen-reader order
- chart summaries
- progress descriptions
- image descriptions where useful
- focus behavior
- reduced motion
- accessible error states

The polished UI must remain usable.

Performance

Avoid:

- unnecessary rebuilds
- oversized decoded images
- uncontrolled animation loops
- excessive blur
- expensive shadows everywhere
- large uncompressed assets
- deeply nested scroll views
- layout thrashing
- loading full-resolution profile images unnecessarily

Use:

- RepaintBoundary where justified
- image resizing
- cached assets
- efficient lists
- const widgets
- isolated animation controllers
- restrained effects

Testing requirements

Add or update tests for:

- shell navigation
- active tab state
- Home hero
- progress display
- quick logging
- weather-enabled dashboard
- weather-disabled dashboard
- weather failure state
- profile navigation
- avatar menu
- local photo selection abstraction
- photo replacement
- photo removal
- shark fallback
- profile editing without onboarding
- progress charts
- challenge cards
- Bottle Bingo interaction
- Bottle Bingo persistence
- legal navigation
- legal section structure
- startup reduced motion
- small-screen rendering
- large text scaling
- SafeArea behavior
- existing hydration logging
- settings access
- no fake sign-out behavior

Do not require a native device for standard unit and widget tests.

Validation

Run:

```text
flutter pub get
dart format --set-exit-if-changed .
flutter analyze
flutter test
flutter build web --release
flutter doctor -v
```

Run Android builds only if the Android SDK is available:

flutter build apk --release
flutter build appbundle --release

Do not falsely claim Android build success if the SDK is unavailable.

Do not falsely claim iOS build success without macOS and Xcode.

Check:

asset sizes

dependency changes

plugin registration

Android permissions

iOS permission descriptions

responsive layout

generated files

secret leakage

hard-coded absolute paths

Completion standard

This work is not complete if Codex only:

changes the startup screen

recolors existing cards

adds a bottom navigation bar without redesigning screens

adds gradients

changes typography

adds a profile page that still resembles onboarding

creates legal pages with a few paragraphs

adds profile photo picking without a real profile experience

adds shark images without meaningful state behavior

claims polish based only on screenshots

The work is complete only when:

Home has a clear character-led dashboard

navigation is polished

Profile is a real destination

local photo selection works

Progress feels useful

Challenges feel active

Bottle Bingo is interactive

Coach feels contextual

Settings are secondary

legal content is credible

weather personalization is visible

shark states are meaningful

startup is polished

layout is responsive

accessibility is maintained

tests pass

existing hydration behavior remains intact

Final report

At completion, return:

Audit findings

current UI problems

current navigation problems

current design-system weaknesses

current asset limitations

current responsive risks

current accessibility risks

Design system

colors

typography

spacing

radii

surfaces

motion

iconography

chart treatment

semantic states

Screen implementation

For each screen:

old problem

new structure

new visual hierarchy

new interactions

reused logic

new components

accessibility behavior

responsive behavior

Cover:

startup

onboarding

Home

Challenges

Progress

Coach

Profile

Settings

legal pages

Shark companion

states added

state-selection logic

asset mapping

animation behavior

reduced-motion behavior

limitations

Weather personalization

visible differences

fallback states

privacy behavior

error behavior

Files

every file changed

every file added

every file removed

reason for each

Tests

every test added

every test updated

total passing

failures

skipped native validation

Validation

every command run

result

warnings

blocked builds

manual checks required

Remaining limitations

asset limitations

device validation

iOS validation

animation limitations

legal review

accessibility review

performance checks

Do not stage, commit, push, branch, publish, or release.

The goal is not to make Hydrion merely functional.

The goal is to make Hydrion feel like a polished, recognizable, emotionally engaging mobile product with enough taste and structure to stand beside modern health, wellness, and gamified applications.